Scheduling Real-Time Parallel Applications in SaaS Clouds in the Presence of Transient Software Failures

Georgios L. Stavrinides and Helen D. Karatza
Department of Informatics
Aristotle University of Thessaloniki
54124 Thessaloniki, Greece
Email: {gstavrin, karatza}@csd.auth.gr

***Abstract*—One of the major challenges in a Software as a Service (SaaS) cloud, is the fault-tolerant and cost-effective scheduling and execution of the end-user applications within strict time constraints, in order to provide results of acceptable quality. Towards this direction, we investigate the performance of strategies for the scheduling of fine-grained parallel applications in a SaaS cloud in the presence of transient software failures, which may occur during the execution of the applications. We combine and compare by simulation different techniques that incorporate application-directed checkpointing and approximate computations, in an attempt to: (a) provide resilience against temporary software failures, (b) guarantee that all applications will meet their deadline, (c) provide application results of as high quality as possible and (d) minimize the monetary cost charged to the end-users for the execution of their applications.**

***Index Terms*—Gang scheduling, software failures, application-directed checkpoints, approximate computations, quality of service, execution cost, SaaS clouds.**

I. Introduction

With the increasing popularity of cloud computing, many vendors switched from the traditional software distribution model, which required the installation of the software on the client's on-premises servers, to the *Software as a Service (SaaS)* cloud computing model [1]. According to this approach, the software is hosted by the vendor and made available to the customers over the Internet. Salesforce, Google Apps, Microsoft Online Services and Adobe Creative Cloud are examples of this model.

The SaaS model provides many benefits, such as easier administration, the software is always up to date and it can be accessed from different locations and devices. Most importantly, consumers are relieved from the burden of acquiring, maintaining and monitoring expensive hardware and software infrastructure [2], [3]. In a SaaS cloud, the end-users submit their applications for execution, without having any control on the host environment. Moreover, a *multi-tenant* model is usually employed, where applications of different end-users may share the same virtual machines (VMs) [4], [5], [6].

In case an application has a *deadline* within which it must finish its execution, it is referred to as *real-time* [7], [8], [9]. Since it is often more desirable for a real-time application to provide an approximate result by its deadline, than a precise result late, Lin et al. proposed the *approximate computations* technique [10]. According to this approach, a real-time application is allowed to return intermediate, approximate results of poorer, but still acceptable quality, when its deadline cannot be met. To achieve this, it is assumed that the application is *monotone*. That is, the quality of its results is improved as more time is spent to produce them. Numerical computation, statistical estimation and prediction, video and voice transmission, heuristic search and database query processing are some examples of monotone applications [11], [12].

Application-directed checkpointing is the most widely used fault-tolerance approach, as opposed to *system-directed checkpointing*, since it is more practical, easier to implement, and system independent [13]. According to this technique, an application is responsible for checkpointing its own progress periodically, at regular intervals during its execution. When a failure occurs, the application is rolled back to its last generated checkpoint.

A. Motivation

The applications submitted to a SaaS cloud by the end-users usually feature *fine-grained parallelism*. That is, each application consists of frequently communicating parallel tasks, which form a *gang*. An effective way to schedule such applications is *gang scheduling* [14]. According to this approach, the parallel tasks of an application are scheduled and executed simultaneously on different VMs. That is, all of the tasks of the application start execution at the same time. This way, the risk of a task waiting to communicate with another task that is currently not running is avoided. Without gang scheduling, the synchronization of all of the component tasks of an application would require more context switches and thus additional overhead [15].

A *Service Level Agreement (SLA)* is a contract between the SaaS cloud provider and the end-users and defines the type of the provided service, the quality of service requirements (e.g. time constraints), the pricing scheme and the cost for the provided service [16]. Therefore, one of the major challenges in a SaaS cloud, is the fault-tolerant and cost-effective

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 07,2025 at 18:08:09 UTC from IEEE Xplore. Restrictions apply.

scheduling and execution of the end-user parallel applications within strict time constraints, in order to provide results of acceptable quality.

B. Contribution

Towards this direction, we investigate the performance of strategies for the scheduling of fine-grained parallel applications in a SaaS cloud in the presence of transient software failures, which may occur during the execution of the applications. We combine and compare by simulation different techniques that incorporate application-directed checkpointing and approximate computations, in an attempt to: (a) provide resilience against temporary software failures, (b) guarantee that all applications will meet their deadline, (c) provide application results of as high quality as possible and (d) minimize the monetary cost charged to the end-users for the execution of their applications. To the best of our knowledge, none of the scheduling approaches proposed in the literature address all of the above issues in the context of a SaaS cloud.

The remainder of this paper is organized as follows: Section II presents an overview of relevant literature, while Section III gives a description of the target SaaS cloud, the workload model, the employed application-directed checkpointing and approximate computations techniques, as well as the cloud pricing model. Section IV describes the scheduling objectives and the investigated scheduling strategies and Section V defines the performance metrics, describes the experimental setup and presents and analyzes the simulation results. Finally, Section VI concludes the paper, providing future research directions.

II. Related Work

Gang scheduling has been studied extensively in the literature [17], [18], [19], [20]. Karatza in [21] studies via a simulation model the performance of gang scheduling of parallel applications in a distributed system, in the presence of soft real-time periodic applications that can tolerate deadline misses by bounded amounts. The simulation results indicate that the performance of the investigated gang scheduling policies depends on the workload, the period of the soft real-time applications, as well as the size of the bounded tardiness.

Han et al. in [22] propose a fault-tolerant scheduling policy for real-time periodic applications with possible software faults. Their approach is based on the last chance strategy and employs approximate computations. It is assumed that each application has two versions, a primary and an alternate version. The primary version is more complex and thus more prone to software failures, while the alternate version is simpler and provides approximate, but still acceptable results. The proposed strategy attempts to execute the primary versions of the applications first. An alternate version of an application is only executed when there is insufficient time to execute its primary version or when a software failure occurs during the execution of the primary version.

Genez et al. in [23] propose an algorithm in an attempt to solve the problem of scheduling an application in a SaaS cloud with two SLA levels: (a) the first SLA level is between the customer and the SaaS provider and concerns the deadline within which the application must be completed and (b) the second SLA level is between the SaaS provider and other IaaS providers and concerns the characteristics and pricing of the leased and on-demand VMs on which the application of the end-user will be executed. The purpose of the proposed algorithm is to find a feasible mapping between the tasks of the application and the VMs from multiple IaaS providers, so that the total monetary cost is minimized and the total execution time of the application is at most equal to its deadline.

However, none of the above approaches tackle all of the scheduling issues addressed in this paper: the resilience against transient software failures, as well as the timeliness, result precision and execution cost of the applications. Moreover, the work presented in this paper differs from our previous work in [19] and [24], in the following aspects:

(a) the system under study is a SaaS cloud and therefore, along with the other scheduling objectives, we aim at the minimization of the monetary cost charged to the end-users,
(b) we adopt a more realistic model regarding application-directed checkpointing, where the checkpointing overhead is not considered negligible and
(c) we examine different combinations of the baseline algorithm with application-directed checkpoints and/ or approximate computations.

III. Problem Formulation

A. System Model

We investigate a SaaS cloud consisting of V homogeneous VMs. It is assumed that the VMs are fully connected by a virtual network and that the communication between them is contention-free. Each VM consists of a single-core vCPU with service rate μ and has its own memory. That is, it serves its own local queue of tasks. It is assumed that the VMs in the SaaS cloud under study are multi-tenant, i.e. each VM processes tasks of applications submitted by different users. The end-user applications first arrive at a VM acting as a central scheduler [25], responsible for scheduling the application tasks to the other VMs of the cloud. The target SaaS cloud is shown in Figure 1.

B. Workload Model

Multiple end-users submit their applications to the SaaS cloud dynamically. Therefore, it is assumed that the applications arrive in a Poisson stream with rate λ [15]. Each application features fine-grained parallelism, consisting of N parallel, non-preemptible, frequently communicating tasks, requiring the same *service time* S. The parallel service time S of each application is considered to be exponential with mean $1/\mu$, where μ is the service rate of the VMs.

Any communication latencies between an application's tasks are assumed to be included in the application's (parallel) service time S. Each application has a firm *deadline* D, after which its results would be useless. Therefore, when an

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 07,2025 at 18:08:09 UTC from IEEE Xplore. Restrictions apply.

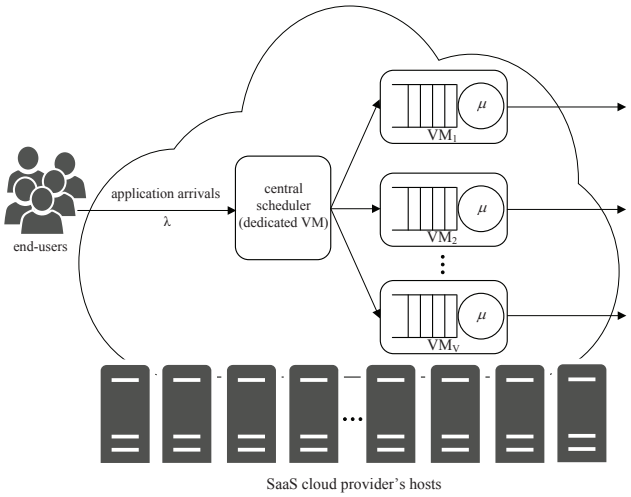


Fig. 1. The SaaS cloud under study, where the end-users submit their applications for execution.

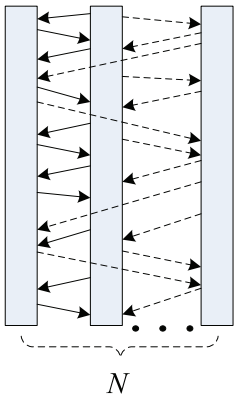


Fig. 2. An example of an application featuring fine-grained parallelism, consisting of N parallel, frequently communicating tasks.

application fails to complete its execution within its deadline, it is considered lost. This is a reasonable assumption, as firm deadlines are frequently considered in the relevant literature [23], [26], as well as in real-world SaaS cloud applications, such as those concerning financial services. The *makespan* (i.e. the total execution time) M of an application is defined as:

$$M = AFT - AST \tag{1}$$

where AFT is the actual finish time of the application, whereas AST is its actual start time. An example of an application with N parallel tasks is illustrated in Figure 2.

Since in gang scheduling all of the tasks of an application must start execution at the same time, some VMs may be idle, even when their queue is not empty. Specifically, a task at the head of the queue of an idle VM may be waiting for the other tasks of its application (which may not be at the head of their assigned queues) to be able to start execution.

C. Software Failures and Application-Directed Checkpoints

During the execution of an application, a software failure may occur due to a transient software fault (e.g. a temporary exception, such as a timeout or a deadlock), with probability FP, affecting one or more of its parallel tasks. It is assumed that no more than one software failure may occur during the execution of an application. When a failure occurs, all of the parallel tasks of the application are rolled back to their beginning and the application resumes execution. This action is considered to cause a *rollback overhead* RB, which is given by:

$$RB = RBF \cdot S \tag{2}$$

where RBF is the *rollback overhead factor* of the application and S is its parallel service time.

In order to tackle the problem of rolling back to the beginning of an application when a failure occurs, we consider that application-directed checkpoints could be utilized. According to this approach, an application is responsible for checkpointing its own progress periodically, at regular intervals during its execution. Specifically, all of the parallel tasks of the application save their current state periodically, in a synchronous manner. Consequently, if a failure occurs, the application is not rolled back to its beginning, but to its last generated checkpoint. The frequency of these application-directed checkpoints is determined by a *checkpointing interval* CPI. The *checkpointing overhead* CP of each checkpoint is defined as:

$$CP = CPF \cdot S \tag{3}$$

where CPF is the *checkpointing overhead factor* of the application.

D. Approximate Computations

In an attempt to improve the performance and enhance the resilience of the SaaS cloud, the technique of approximate computations could be employed. According to this method, each application is considered to be monotone and consists of a *mandatory part* MP, followed by an *optional part* OP, as follows:

$$S = MP + OP \tag{4}$$

where S is the parallel service time of the application.

An application is completed when at least its mandatory part has been completed. The optional part of the application refines the result provided by its mandatory part and can be either fully completed, partially completed or it may be entirely skipped. The discarded fraction of the optional part negatively affects the precision of the application's results.

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 07,2025 at 18:08:09 UTC from IEEE Xplore. Restrictions apply.

Specifically, the *result precision* RP of an application is given by:

$$RP = \frac{S_{actual}}{S} \tag{5}$$

where S_{actual} is the part of the service time that the application actually completed. It is assumed that according to the SLA between the SaaS cloud provider and the end-users, there is a *result precision threshold* RPT, below which an application's results are not acceptable. It is defined as:

$$RPT = \frac{MP}{S} \tag{6}$$

That is, the minimum acceptable result precision is given when only the mandatory part of the application is completed.

In the case of approximate computations, since every application should be able to complete at least its mandatory part before its deadline is reached, we use the notion of the *notification time* NT, as in our previous work in [24]. It is defined as:

$$NT = D - MP \tag{7}$$

where D is the deadline of the application.

Specifically, when an application is waiting for service and its notification time is reached, it must immediately start execution, as there is only available time to execute its mandatory part. Otherwise, the application will definitely fail to complete its mandatory part within its deadline and therefore it will be lost. A notified application can occupy its assigned VMs and start execution only when the VMs are idle or when all of the applications of the tasks in service on the particular VMs have completed their mandatory part. In the latter case, the applications in service are aborted, giving as a result the one produced at that time instant. It is noted that in this case, the notified application will complete only its mandatory part and, in case application-directed checkpoints are utilized, no checkpoints will occur during its execution, as there is not enough time available for checkpointing. On the other hand, if the notified application cannot immediately start execution, it is discarded from the system, because it will definitely miss its deadline.

In case an application in service reaches its deadline and it has completed its mandatory part, then it is not considered lost. On the contrary, it is aborted, giving as a result the one produced at the time of its deadline. In any other case the application is lost.

In the case where application-directed checkpointing is employed and a failure occurs during the execution of an application, if the checkpointed part of the application's service time is greater than its mandatory part, then there is no need for rollback. Instead, the application is aborted, giving as a result the one recorded at its latest checkpoint. If a failure occurs during the execution of a notified application, then the application is lost, since there is no time available to roll it back (it will definitely miss its deadline). In any other case the affected application is rolled back and resumes execution from either its latest checkpoint (in case application-directed checkpointing is used) or its beginning (in case no application-directed checkpointing is employed).

E. Pricing Model

SaaS cloud providers are flexible to use various pricing models for their services, such as a pay-as-you-go or a subscription based model. In this paper, for simplicity reasons and without loss of generality, we consider a pay-as-you-go pricing scheme, according to which each user is charged a fixed amount per time unit for the execution of each of their submitted applications. That is, the monetary cost required for the execution of an application, is proportional to its required makespan (i.e. its total execution time). It is assumed that according to the SLA between the cloud provider and the end-users, there is no cost charged for applications that fail to complete their execution within their deadline and thus they are lost.

IV. Scheduling Strategies

A. Scheduling Objectives

We investigate the performance of strategies for the gang scheduling of parallel applications in the target SaaS cloud, in the presence of transient software failures. We combine and compare different techniques that incorporate application-directed checkpointing and approximate computations, in an attempt to:

- **(a)** provide resilience against temporary software failures,
- **(b)** guarantee that all applications will meet their deadline,
- **(c)** provide application results of as high quality as possible and
- **(d)** minimize the monetary cost charged to the end-users for the execution of their applications.

B. Scheduling Algorithms

In order to meet the above objectives, we employ the following gang scheduling methods:

EDF:

According to this approach (*Earliest Deadline First*), the tasks of an application are placed in the queues of the VMs that can provide the application with the earliest estimated start time. The tasks are placed in the selected queues according to their application's deadline. That is, the task with the earliest deadline has the highest priority for execution. This policy is used as a baseline algorithm, for comparison purposes against the other approaches that utilize application-directed checkpoints and/ or approximate computations.

EDF_RAC:

According to this scheduling policy (*Earliest Deadline First with Restricted Approximate Computations*), the tasks of an application are placed in the queues of the VMs as in the EDF case. However, in

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 07,2025 at 18:08:09 UTC from IEEE Xplore. Restrictions apply.

this case the technique of approximate computations is employed, as described in Subsection III-D.

EDF_FAC:
This method (*Earliest Deadline First with Full Approximate Computations*), is similar to the EDF_RAC policy, but in this case all of the applications that arrive at the SaaS cloud are scheduled to complete only their mandatory part. That is, in this case the result precision of every completed application will be equal to the result precision threshold RPT.

EDF_ADC:
According to this approach (*Earliest Deadline First with Application-Directed Checkpoints*), the tasks of an application are placed in the queues of the VMs as in the EDF case. However, in this case the technique of application-directed checkpointing is employed, as described in Subsection III-C.

EDF_ADC_RAC :
This method (*Earliest Deadline First with Application-Directed Checkpoints and Restricted Approximate Computations*), is based on the EDF policy, utilizing both application-directed checkpointing (as in the EDF_ADC case) and approximate computations (as in the EDF_RAC case).

EDF_ADC_FAC :
This policy (*Earliest Deadline First with Application-Directed Checkpoints and Full Approximate Computations*), is similar to the EDF_ADC_RAC approach, but in this case all of the applications that arrive at the SaaS cloud are scheduled to complete only their mandatory part (as in the EDF_FAC case).

V. Performance Evaluation

A. Performance Metrics

We evaluated the performance of the strategies described in Section IV in terms of the following metrics:

- *Quality of Service indicator* (QoS), which is defined as:

$$QoS = AGR \cdot \overline{RP} \tag{8}$$

AGR is the *application guarantee ratio*, i.e. the ratio of the number of applications that finished within their deadline, over the number of all of the submitted applications in the SaaS cloud, during the observed time period. $\overline{RP}$ is the *average result precision* of all of the completed applications. That is, it is assumed that according to the SLA between the cloud provider and the end-users, both AGR and $\overline{RP}$ have the same importance in terms of the provided quality of service.

- *Cost savings indicator* (CS), which is defined as:

$$CS = 1 - Cost' \tag{9}$$

$Cost'$ is the *normalized average cost per application* in the range [0,1]. Specifically, it is given by:

$$Cost' = \frac{Cost - Cost_{min}}{Cost_{max} - Cost_{min}} \tag{10}$$

where $Cost_{min}$ and $Cost_{max}$ are respectively the minimum and maximum values of the average cost per application, given by all of the examined scheduling policies for a particular set of simulation input parameters. In particular, the *average cost per application* ($Cost$), is defined as:

$$Cost = \overline{M} \cdot PPTU \tag{11}$$

where $\overline{M}$ is the *average makespan* of all of the completed applications, whereas $PPTU$ is the *price per time unit*, according to the pricing model described in Subsection III-E.

- *Overall system performance indicator* (OSP), which denotes the overall performance of the SaaS cloud, in terms of both the quality of service QoS and the cost savings CS indicators. Specifically, it is defined as:

$$OSP = \frac{QoS + CS}{2} \tag{12}$$

The overall system performance indicator OSP can have a value in [0,1]. It is optimal (i.e. $OSP = 1$) when the employed scheduling strategy provides both optimal quality of service (i.e. $QoS = 1$) and maximum cost savings for the end-user (i.e. $CS = 1$).

B. Experimental Setup

The performance of the SaaS cloud for each of the investigated gang scheduling approaches, was evaluated by conducting a series of simulation runs. For this purpose, we implemented our own discrete-event simulation program in C++, tailored to the specific requirements of the particular problem, due to the complexity of the system and the workload model under study. We used synthetic workload, in order for our simulation results to be unbiased.

The input parameters used in our simulation experiments are shown in Table I. In order to investigate the impact of the transient software failures on the performance of the examined scheduling algorithms, we chose a relatively large value for the failure probability parameter ($FP = 0.5$). The value for the checkpointing interval CPI was chosen accordingly. For each set of input parameters, we ran 30 replications of the simulation program, with different seeds of random numbers. Each replication was terminated when 10^6 applications had finished execution. For every mean value, a 95% confidence interval was evaluated. The half-widths of all of the confidence intervals are less than 5% of their respective mean values. We investigated the performance of each scheduling algorithm with respect to the arrival rate λ of the applications. The simulation results are analyzed in the next section.

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 07,2025 at 18:08:09 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SIMULATION INPUT PARAMETERS

Parameter	Value
Number of completed applications	10^6
Number of VMs in the SaaS cloud	$V = 64$
Service rate of the VMs	$\mu = 1$
Application gang size	$N \sim U[1, 64]$
Application arrival rate	$\lambda = \{0.85, 1.35, 1.85\}$
Application relative deadline	$RD \sim U[S, 2S]$
Price per time unit	$PPTU = \$0.01$
Software failure probability	$FP = 0.5$
Rollback overhead factor	$RBF = 0.1$
Checkpointing interval	$CPI = 0.2$
Checkpointing overhead factor	$CPF = 0.05$
Result precision threshold	$RPT = 0.75$
Scheduling strategy	EDF, EDF_RAC, EDF_FAC, EDF_ADC, EDF_ADC_RAC, EDF_ADC_FAC

C. Simulation Results and Discussion

The comparison of all of the scheduling approaches described in Subsection IV-B, in terms of the three performance indicators defined in Subsection V-A, is illustrated in Figures 3, 4 and 5, for various workload conditions. In terms of the quality of service indicator QoS (Figure 3), the EDF_ADC_RAC policy, which combines application-directed checkpoints and approximate computations in a restricted manner, outperforms all of the other scheduling methods. The particular approach gives an average QoS improvement of 46.45%, over the baseline algorithm, EDF. As it can be observed in Figures 6 and 7, even though the EDF_ADC_RAC strategy does not provide the highest application guarantee ratio AGR and the highest average result precision $\overline{RP}$ among the scheduling policies, however, it seems that it handles better the tradeoff between these two conflicting parameters, ultimately providing the highest QoS.

The EDF_ADC policy provides the lowest QoS. The decline in QoS in the particular case is 7.62%, compared to the baseline algorithm, EDF. This approach, even though it always gives the highest possible average result precision ($\overline{RP} = 1$), it provides the lowest application guarantee ratio AGR amongst all of the examined strategies. This can be justified by the fact that it employs application-directed checkpointing, which imposes significant overhead on the service time of each application, without balancing out the additional overhead with a technique that saves execution time, such as the approximate computations approach.

In terms of the cost savings indicator CS (Figure 4), the EDF_FAC strategy outperforms all of the other policies. This is due to the fact that it provides the smallest average application makespan (and thus the lowest cost) compared to the other policies, since in this case all of the applications are scheduled to complete only their mandatory part. Moreover, in this case there is no additional overhead due to checkpoints, since the application-directed checkpointing technique is not employed. The lowest cost savings are provided by the EDF_ADC policy, which as mentioned earlier, also provides the lowest quality of service.

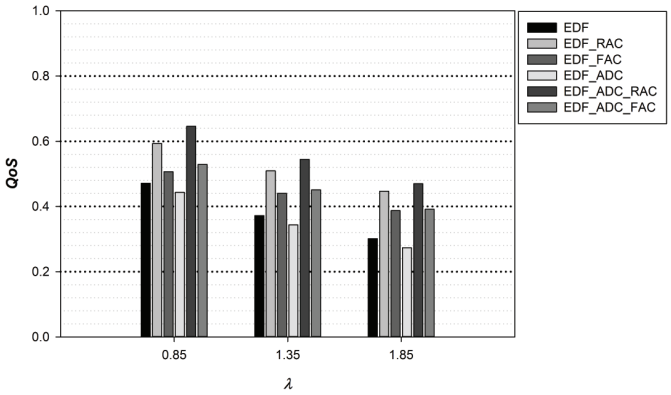


Fig. 3. The Quality of Service indicator (QoS) vs. the arrival rate λ of the applications, for all of the investigated scheduling approaches.

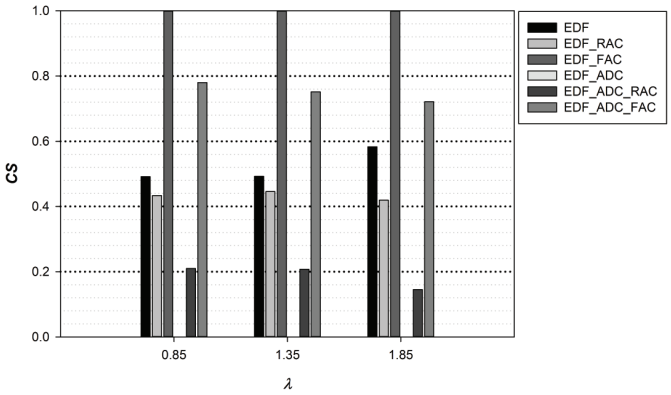


Fig. 4. The cost savings indicator (CS) vs. the arrival rate λ of the applications, for all of the investigated scheduling algorithms.

According to the overall system performance indicator OSP (Figure 5), similar observations can be made as in the cost savings indicator case. That is, the EDF_FAC policy provides the best overall performance, whereas the EDF_ADC strategy the worst. This can be explained by the fact that, since both the quality of service QoS and cost savings CS indicators have the same importance for the calculation of the overall system performance, the substantial cost savings differences amongst the scheduling policies dominate the corresponding differences observed in their provided quality of service.

VI. CONCLUSIONS

In this paper, we investigated the performance of various strategies for the scheduling of fine-grained parallel applications in a SaaS cloud, in the presence of transient software failures. We combined and compared by simulation under various workload conditions different techniques that incorporate application-directed checkpointing and approximate computations, in an attempt to meet several objectives related to the

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 07,2025 at 18:08:09 UTC from IEEE Xplore. Restrictions apply.

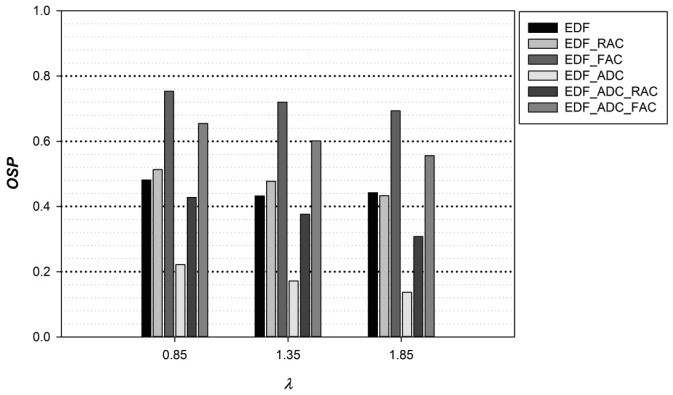


Fig. 5. The overall system performance indicator (OSP) vs. the arrival rate λ of the applications, for all of the investigated scheduling policies.

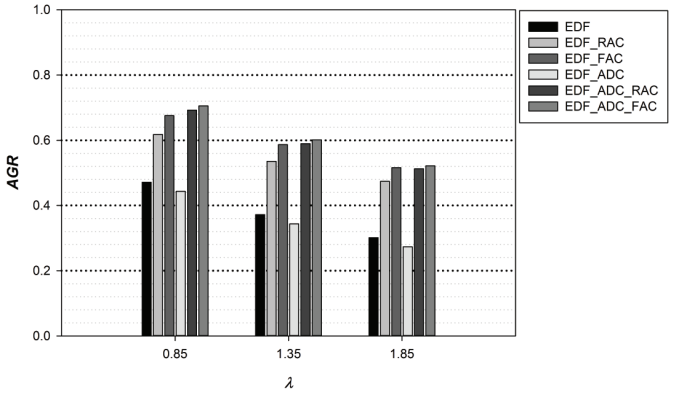


Fig. 6. The application guarantee ratio (AGR) vs. the arrival rate λ of the applications, for all of the investigated scheduling methods.

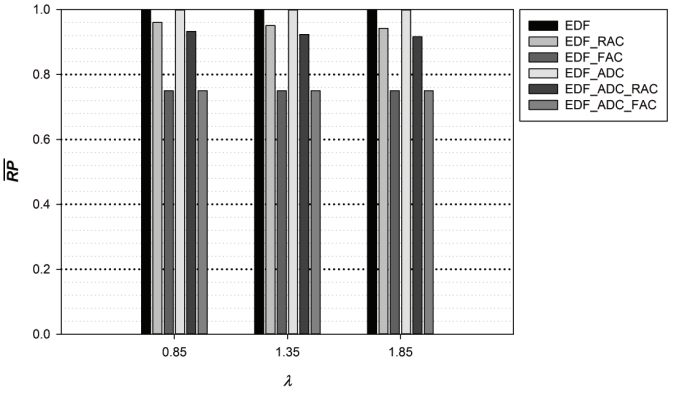


Fig. 7. The average result precision ($\overline{RP}$) vs. the arrival rate λ of the applications, for all of the investigated scheduling methods.

fault-tolerance, quality of service and the minimization of the monetary cost charged to the end-users.

The simulation results indicate that the EDF_ADC_RAC policy provides the highest quality of service amongst the other approaches, whereas the EDF_FAC policy provides the greatest cost savings and the highest overall system performance. Our future work plans include the investigation of different failure probabilities and checkpointing intervals in the framework presented in this paper, since both of these parameters seem to have a great impact on the performance of the employed scheduling policies.

REFERENCES

[1] M. Cusumano, "Cloud computing and SaaS as new computing platforms", *Communications of the ACM*, vol. 53, no. 4, pp. 27–29, Apr. 2010.

[2] A. Beloglazov, J. Abawajy, and R. Buyya, "Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing", *Future Generation Computer Systems*, vol. 28, no. 5, pp. 755–768, May 2012.

[3] R. N. Calheiros and R. Buyya, "Energy-efficient scheduling of urgent bag-of-tasks applications in clouds through DVFS", in *Proceedings of the 6th IEEE International Conference on Cloud Computing Technology and Science (CloudCom'14)*, Singapore, Singapore, Dec. 2014, pp. 342–349.

[4] B. P. Rimal, E. Choi, and I. Lumb, "A taxonomy and survey of cloud computing systems", in *Proceedings of the 5th International Joint Conference on INC, IMS and IDC (NCM'09)*, Seoul, Korea, Aug. 2009, pp. 44–51.

[5] C. N. Höfer and G. Karagiannis, "Cloud computing services: taxonomy and comparison", *Journal of Internet Services and Applications*, vol. 2, no. 2, pp. 81–94, Jun. 2011.

[6] G. L. Stavrinides and H. D. Karatza, "A cost-effective and QoS-aware approach to scheduling real-time workflow applications in PaaS and SaaS clouds", in *Proceedings of the 3rd International Conference on Future Internet of Things and Cloud (FiCloud'15)*, Rome, Italy, Aug. 2015, pp. 231–239.

[7] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed., Springer, 2011.

[8] G. L. Stavrinides and H. D. Karatza, "The impact of resource heterogeneity on the timeliness of hard real-time complex jobs", in *Proceedings of the 7th International Conference on PErvasive Technologies Related to Assistive Environments (PETRA'14)*, Rhodes, Greece, May 2014, pp. 65:1–65:8.

[9] G. L. Stavrinides and H. D. Karatza, "Scheduling real-time jobs in distributed systems - simulation and performance analysis", in *Proceedings of the 1st International Workshop on Sustainable Ultrascale Computing Systems (NESUS 2014)*, Porto, Portugal, Aug. 2014, pp. 13–18.

[10] K. J. Lin, S. Natarajan, and J. W. S. Liu, "Imprecise results: utilizing partial computations in real-time systems", in *Proceedings of the 8th IEEE Real-Time Systems Symposium (RTSS'87)*, San Jose, CA, Dec. 1987, pp. 210–217.

[11] G. L. Stavrinides and H. D. Karatza, "Scheduling multiple task graphs in heterogeneous distributed real-time systems by exploiting schedule holes with bin packing techniques", *Simulation Modelling Practice and Theory*, vol. 19, no. 1, pp. 540–552, Jan. 2011.

[12] G. L. Stavrinides and H. D. Karatza, "Scheduling real-time DAGs in heterogeneous clusters by combining imprecise computations and bin packing techniques for the exploitation of schedule holes", *Future Generation Computer Systems*, vol. 28, no. 7, pp. 977–988, Jul. 2012.

[13] R. A. Oldfield, S. Arunagiri, P. J. Teller, S. Seelam, M. R. Varela, R. Riesen, and P. C. Roth, "Modeling the impact of checkpoints on next-generation systems", in *Proceedings of the 24th IEEE Conference on Mass Storage Systems and Technologies (MSST'07)*, San Diego, CA, Sep. 2007, pp. 30–46.

[14] H. D. Karatza, "The impact of critical sporadic jobs on gang scheduling performance in distributed systems", *Simulation: Transactions of the Society for Modeling and Simulation International*, Special Issue on Performance Evaluation of Computer and Telecommunication Systems, vol. 84, no. 2–3, pp. 89–102, Feb. 2008.

[15] H. D. Karatza, "Performance of gang scheduling strategies in a parallel system", *Simulation Modelling Practice and Theory*, vol. 17, no. 2, pp. 430–441, Feb. 2009.

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 07,2025 at 18:08:09 UTC from IEEE Xplore. Restrictions apply.

[16] Y. Zhao, R. N. Calheiros, G. Gange, K. Ramamohanarao, and R. Buyya, "SLA-based resource scheduling for big data analytics as a service in cloud computing environments", in *Proceedings of the 44th International Conference on Parallel Processing (ICPP'15)*, Beijing, China, Sep. 2015, pp. 510–519.

[17] Y. Zhang, A. Yang, A. Sivasubramaniam, and J. Moreira, "Gang scheduling extensions for I/O intensive workloads", in *Proceedings of the 9th International Workshop on Job Scheduling Strategies for Parallel Processing (JSSPP'03)*, Seattle, WA, Jun. 2003, pp. 183–207.

[18] H. D. Karatza, "Scheduling gangs in a distributed system", *International Journal of Simulation: Systems, Science and Technology*, vol. 7, no. 1. pp. 15–22, Jan. 2006.

[19] G. L. Stavrinides and H. D. Karatza, "Performance evaluation of gang scheduling in distributed real-time systems with possible software faults", in *Proceedings of the 2008 International Symposium on Performance Evaluation of Computer and Telecommunication Systems (SPECTS'08)*, Edinburgh, UK, Jun. 2008, pp. 1–7.

[20] V. Manickam and A. Aravind, "A fair and efficient gang scheduling algorithm for multicore processors", in *Proceedings of the 6th International Conference on Information Processing (ICIP'12)*, Bangalore, India, Aug. 2012, pp. 467–476.

[21] H. D. Karatza, "Scheduling jobs with different characteristics in distributed systems", in *Proceedings of the 2014 International Conference on Computer, Information and Telecommunication Systems (CITS'14)*, Jeju Island, South Korea, Jul. 2014, pp. 1–5.

[22] C. C. Han, K. G. Shin, and J. Wu, "A fault-tolerant scheduling algorithm for real-time periodic tasks with possible software faults", *IEEE Transactions on Computers*, vol. 52, no. 3, pp. 362–372, Mar. 2003.

[23] T. A. L. Genez, L. F. Bittencourt, and E. R. M. Madeira, "Workflow scheduling for SaaS/ PaaS cloud providers considering two SLA levels", in *Proceedings of the 2012 IEEE Network Operations and Management Symposium (NOMS'12)*, Maui, HI, Apr. 2012, pp. 906–912.

[24] G. L. Stavrinides and H. D. Karatza, "Fault-tolerant gang scheduling in distributed real-time systems utilizing imprecise computations", *Simulation: Transactions of the Society for Modeling and Simulation International*, Special Issue on Performance Evaluation of Computer and Telecommunication Systems, vol. 85, no. 8, pp. 525–536, Aug. 2009.

[25] X. Zhu, X. Qin, and M. Qiu, "QoS-aware fault-tolerant scheduling for real-time tasks on heterogeneous clusters", *IEEE Trans. Comp.*, vol. 60, no. 6, pp. 800–812, Jun. 2011.

[26] K. H. Kim, A. Beloglazov, and R. Buyya, "Power-aware provisioning of cloud resources for real-time services", in *Proceedings of the 7th International Workshop on Middleware for Grids, Clouds and e-Science (MGC'09)*, Urbana-Champaign, IL, Nov. 2009, pp. 1–6.

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 07,2025 at 18:08:09 UTC from IEEE Xplore. Restrictions apply.